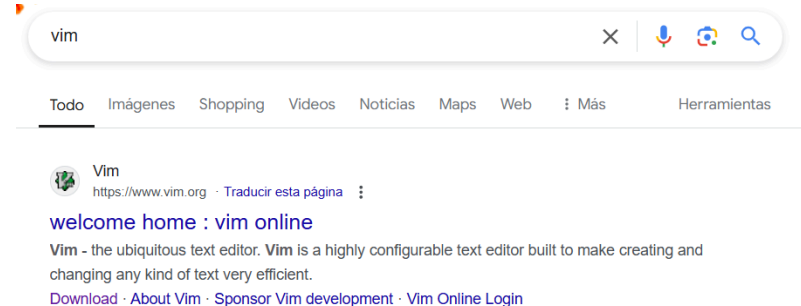


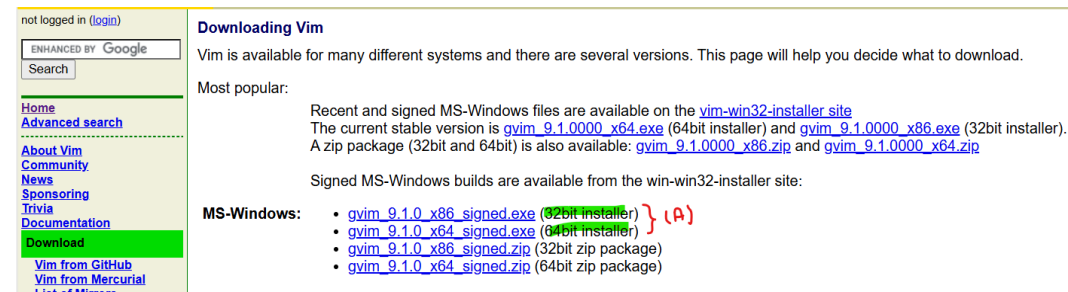
Anexo A

Instalación de Vim

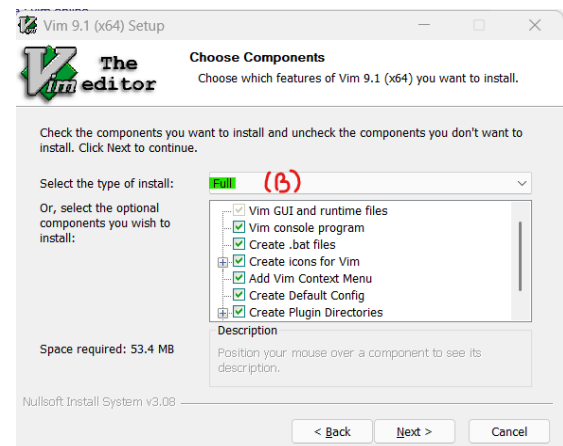
En el explorador busque la página del proyecto VIM:



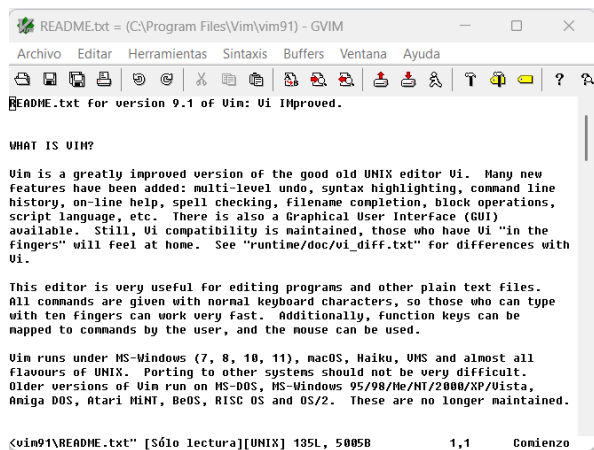
En la sección de descargas (Download), elija la opción más adecuada para su sistema:



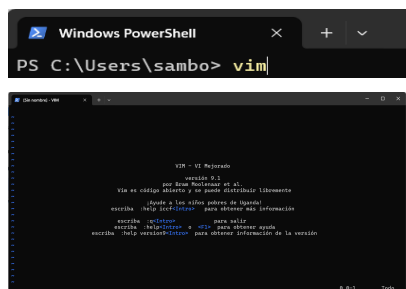
Se muestra en (A) las opciones para sistemas de 32 y 64 bits respectivamente, este instalador colocará una versión GUI para usar el editor en Windows, aunque una instalación mínima es suficiente se recomienda la versión completa si se cuenta con espacio suficiente.



Se preguntará al final si se desea leer el archivo “readme” para conocer detalles sobre la versión que se ha instalado, puede ser una buena oportunidad para ver la GUI del editor:



Es posible utilizar la versión de Vim en la ventana de CMD y Powershell de Windows, en dichas ventanas simplemente teclee `vim` y con eso invocará al editor:



Listo ya puede usar Vim en sus terminales.

Anexo B

Instalación de MinGW64

Visite la siguiente página web para información sobre el proceso de instalación descrito en esta sección:

[Using GCC with MinGW](#)

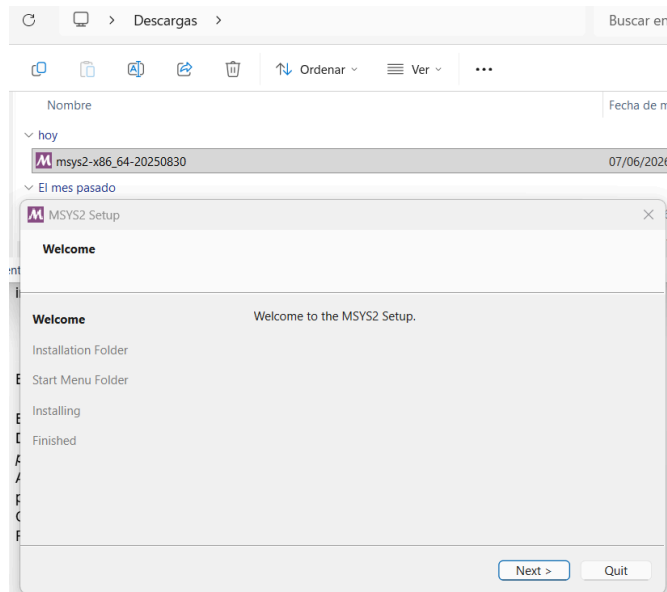
Como paso previo requiere la instalación del editor VS Code con las extensiones *C/C++* *Intellisense*, para instalarlas busquelas en el listado de la ventana de extensiones (`Ctrl+Shift+X`) del editor.

Instalación de las herramientas de MinGW64

De la página mostrada anteriormente del enlace [direct link to the installer](#) descargue el instalador para los paquetes requeridos.

- 1 You can download the latest installer from the MSYS2 page or use this [direct link to the installer](#).

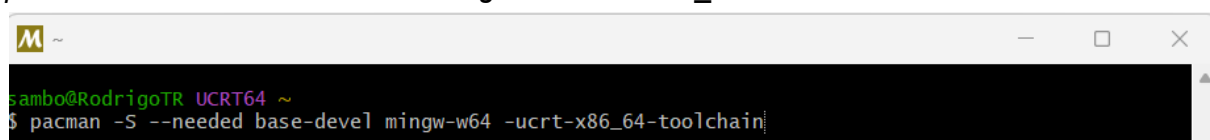
Ejecute el instalador y siga las instrucciones indicadas en el proceso de instalación



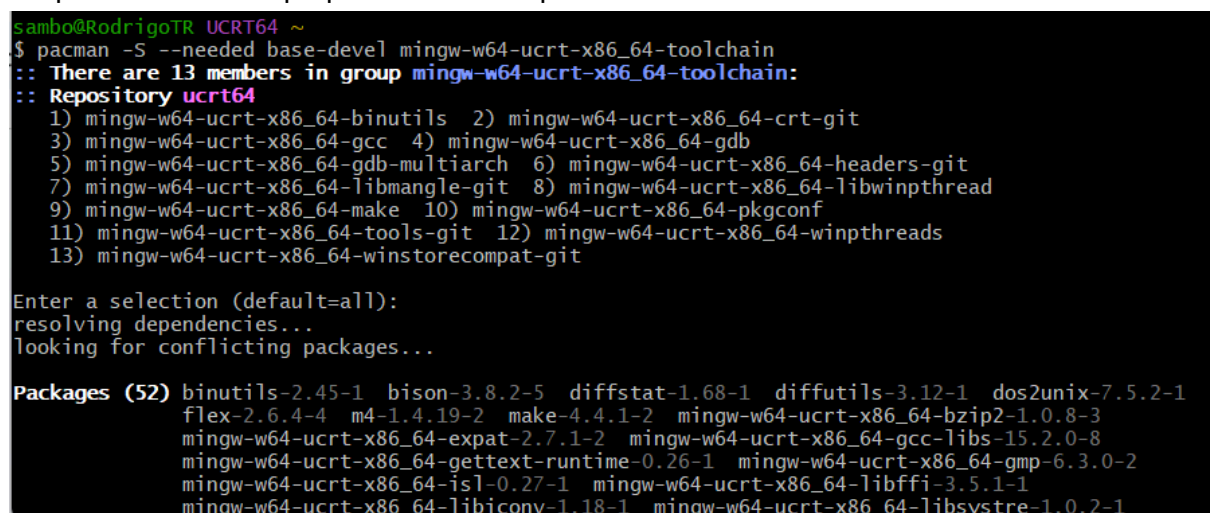
El instalador ejecutará una pantalla para completar la instalación

Donde ejecutará los siguientes comandos:

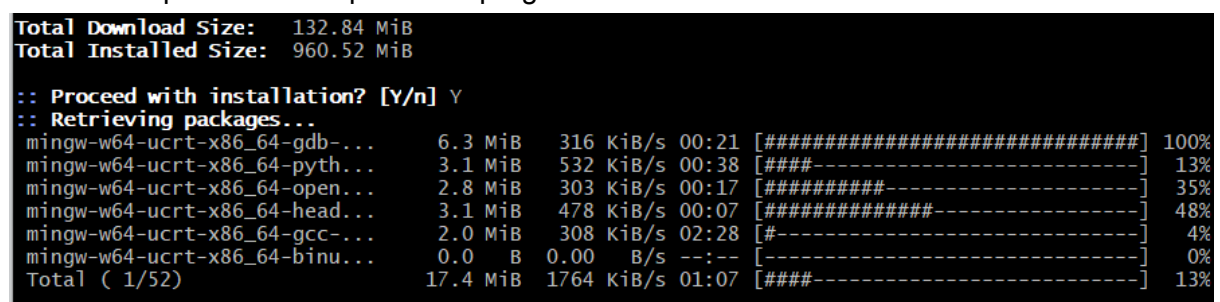
```
pacman -S --needed base-devel mingw-w64-ucrt-x86_64-toolchain
```



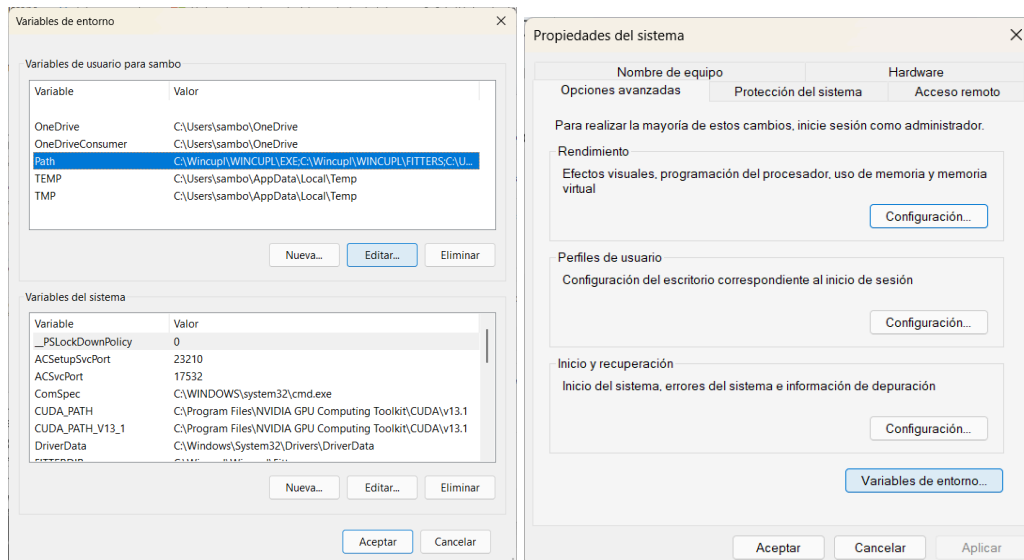
Acepta el número de paquetes a instalar por default



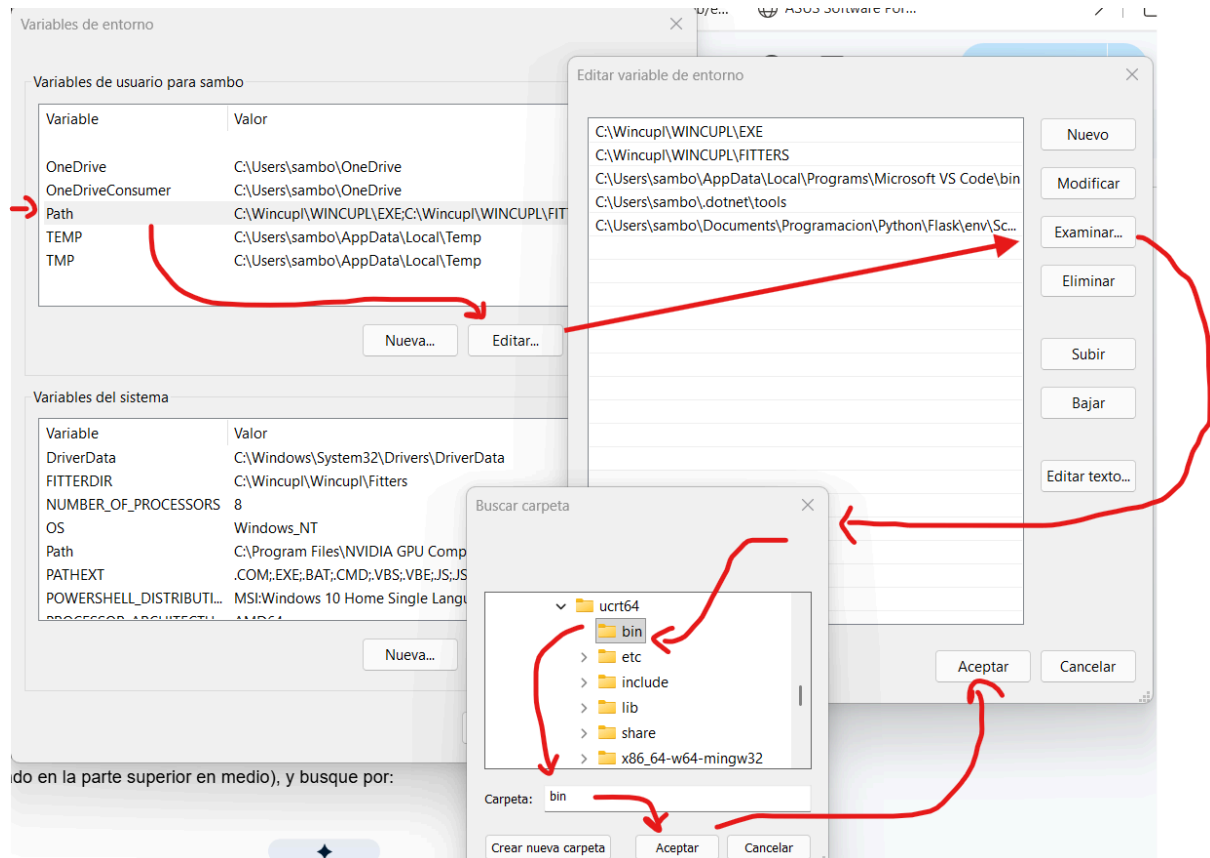
Y escribe Y para cada aceptar cada pregunta durante la instalación



En la sección de inicio de Windows escriba:



En la sección de variables de usuario agregamos la ubicación de la herramienta *ucrt64* (ubicada en: *C:\msys64\ucrt64\bin*)



Revise en la terminal que la instalación se realizó correctamente con el comando:
gcc --version

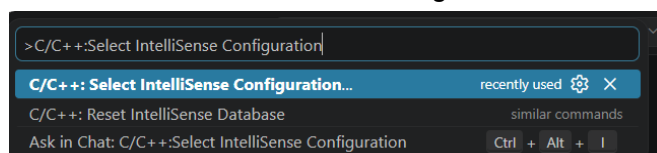
```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\sambo> gcc --version
gcc.exe (Rev8, Built by MSYS2 project) 15.2.0
Copyright (C) 2025 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.
```

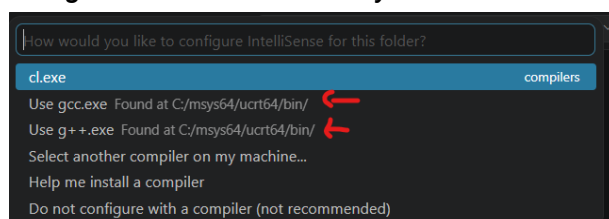
Adicionalmente en VS code en la ventana de editor seleccione el CLI de extensiones (ubicado en la parte superior en medio), y busque por:

>C/C++:Select IntelliSense Configuration



Si fue configurado correctamente usted verá listado:

Use gcc.exe Found at C:\msys64\bin\...

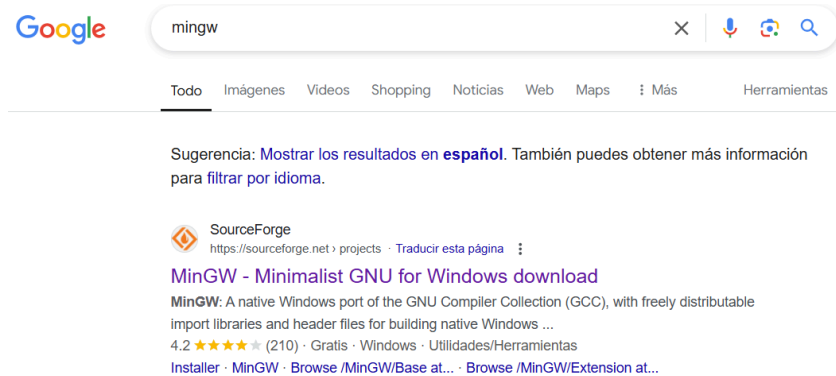


Anexo C

Instalación de MinGW (OBSOLETO INSTALAR MinGW PARA 64 BITS ANEXO B)

En el explorador buscamos MinGW y elegimos la opción de SourceForge.net:

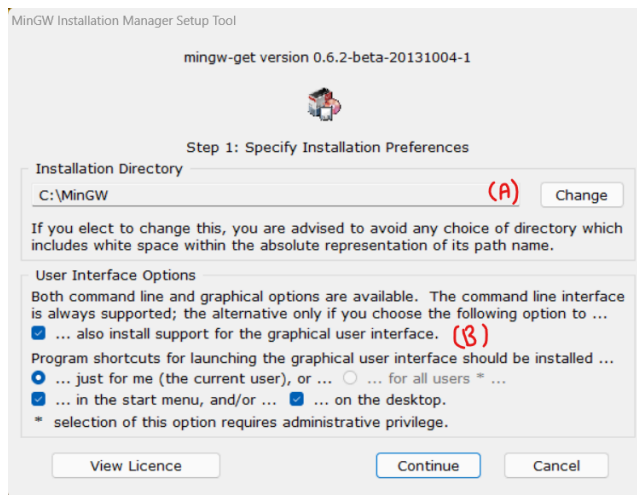
[MinGW - Minimalist GNU for Windows download | SourceForge.net](https://sourceforge.net/projects/mingw/)



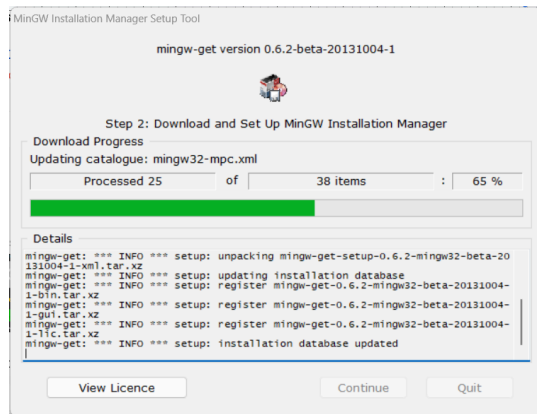
Descargamos el Instalador y lo ejecutamos en con permisos de administrador:



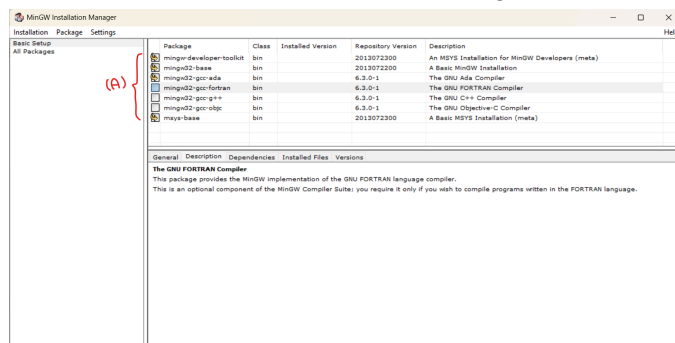
La siguiente figura muestra información sobre la instalación, (A) indica el directorio donde se estará el programa, (B) si queremos instalar la GUI o solo el conjunto de programas para el CLI (terminal).



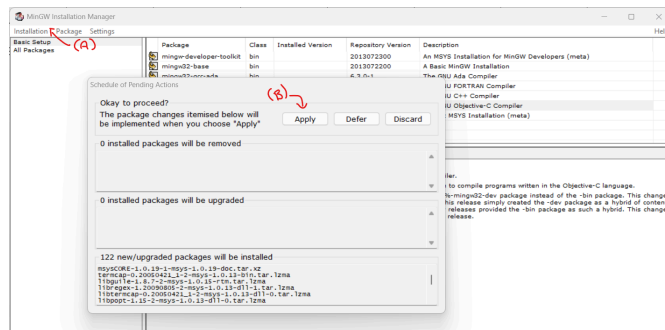
Comenzará a descargar los archivos de la instalación y los depositara en la carpeta antes mencionada:



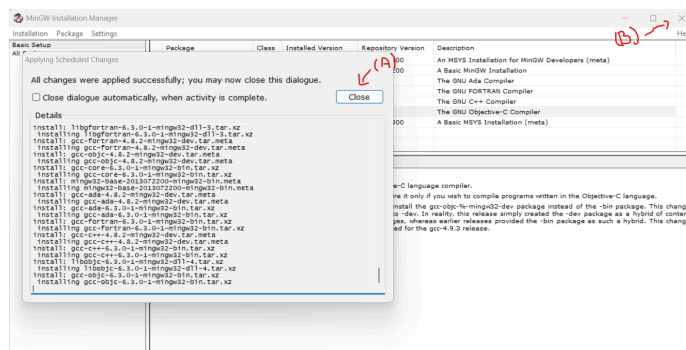
Al terminar nos pedirá que continuemos y nos indicará que debemos marcar los paquetes que instalarán (A), se recomienda elegirlos todos:



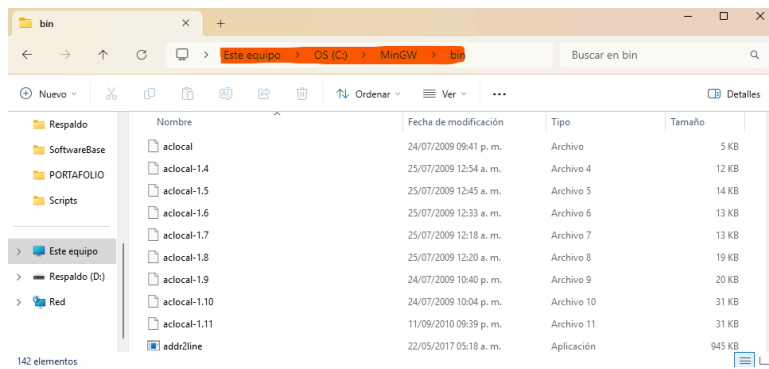
En el menú (A) elegimos la opción “apply changes” y en la ventana (B) presionamos el botón “Apply”, esperamos que termine la instalación:



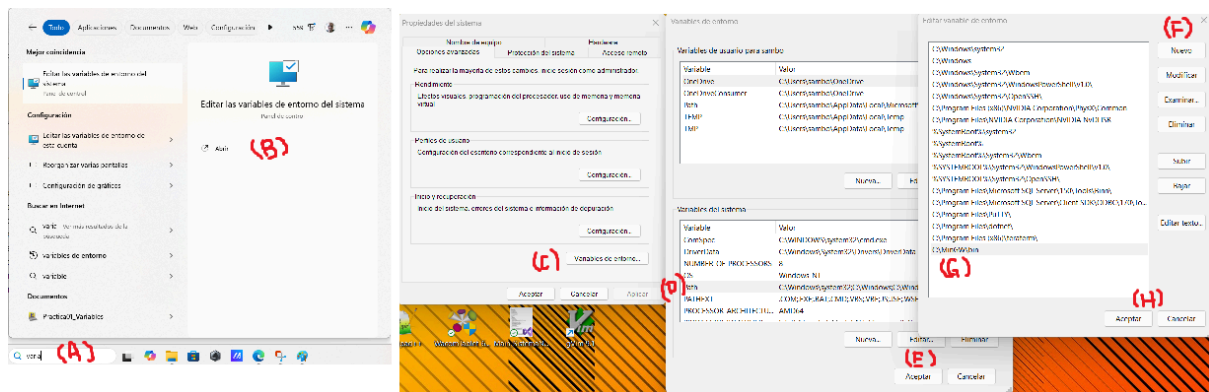
Una vez terminada la instalación cerramos el cuadro de diálogo (A) y la ventana (B):



En la ventana del Explorador de Archivos, nos ubicamos en la dirección donde se instalaron los archivos (indicada al inicio de la instalación), dentro de la carpeta `bin`:



Y copiamos la dirección al portapapeles (clic con el botón derecho del ratón), esto para agregarla a la dirección de búsqueda de programas ejecutables en las variables de entorno de Windows:



En el cuadro de búsqueda de Windows, escribimos *Variables de Ambiente* (A), ejecutamos la aplicación (B), presionamos el botón *Variables de Entorno* (C), elegimos del cuadro de texto la opción *Path* (D), en la ventana el botón (F) y en (G) pegamos la dirección que copiamos del explorador de Windows y presionamos el botón (H) para terminar (damos click en Aceptar en las ventanas anteriores).

En la terminal de PowerShell tecleamos el comando `g++ --version`, para verificar la instalación:

```
Windows PowerShell
Copyright (C) Microsoft Corporation. Todos los derechos reservados.

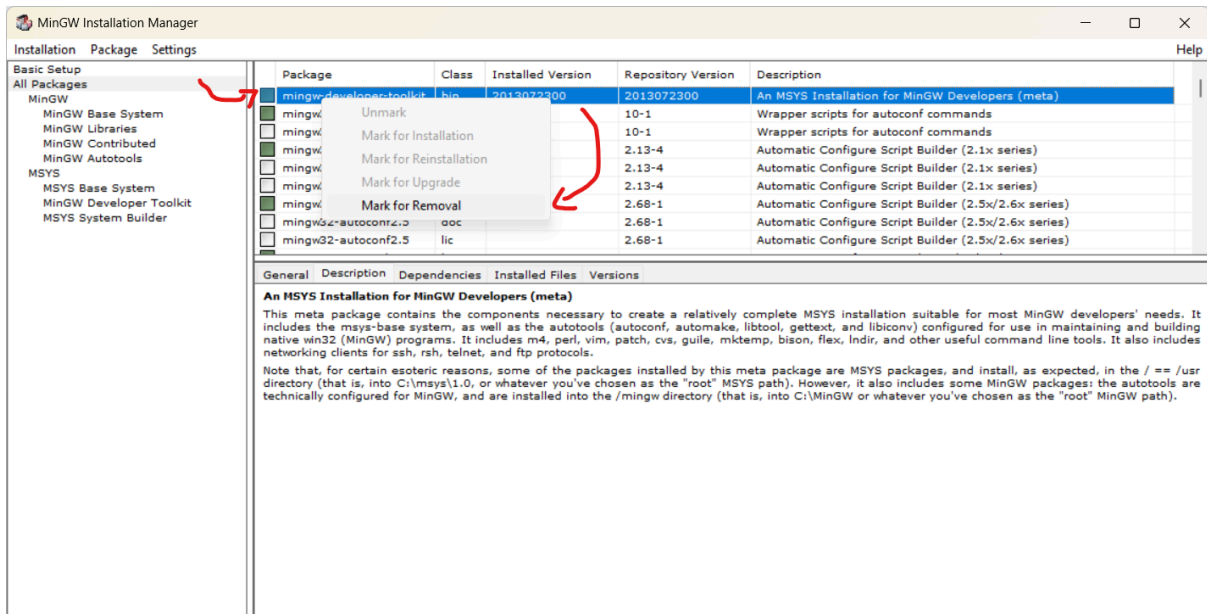
Instale la versión más reciente de PowerShell para obtener nuevas características y mejoras. https://aka.ms/PSWindows

PS C:\Users\sambo> g++ --version (A)
g++.exe (MinGW.org GCC-6.3.0-1) 6.3.0
Copyright (C) 2016 Free Software Foundation, Inc.
This is free software; see the source for copying conditions. There is NO
warranty; not even for MERCHANTABILITY or FITNESS FOR A PARTICULAR PURPOSE.

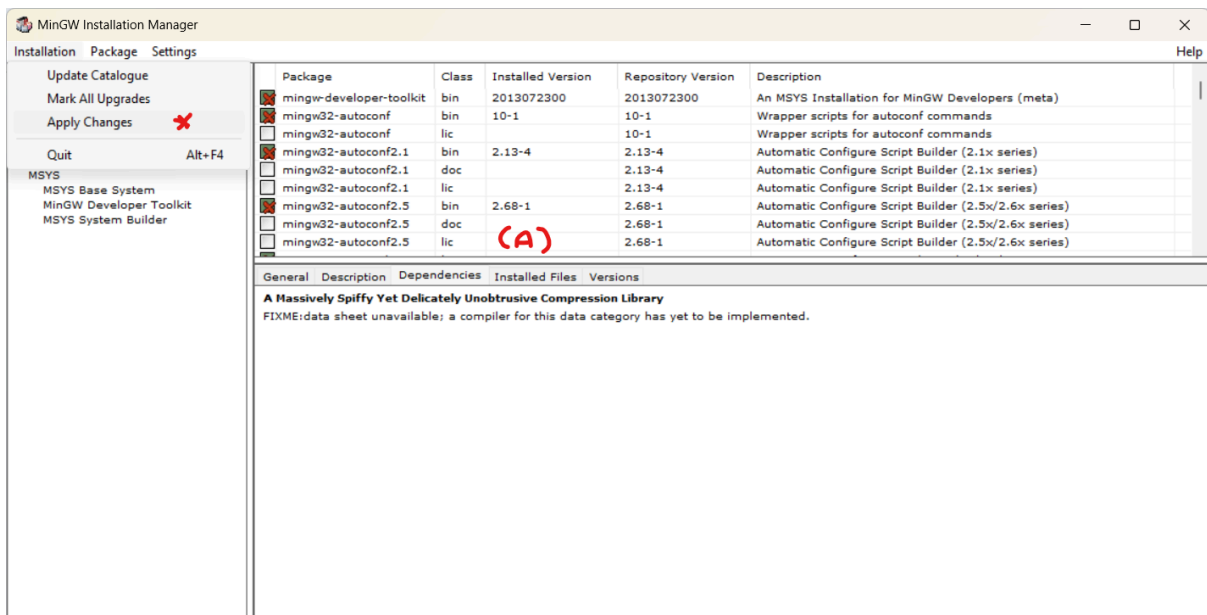
PS C:\Users\sambo>
```

Si nos indica la versión entonces hemos instalado el software correctamente.

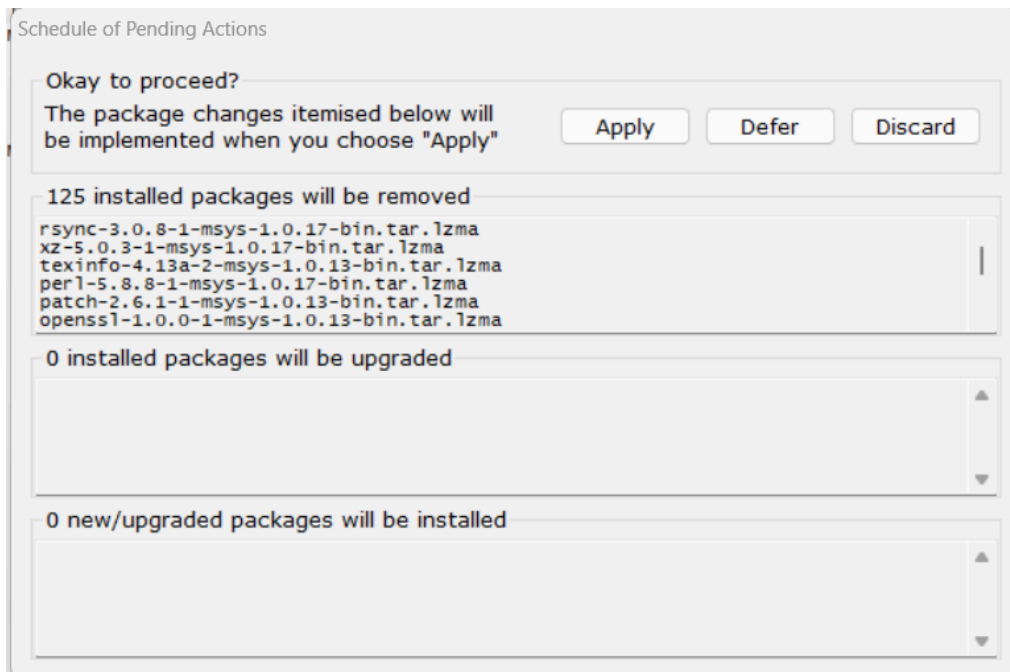
En caso de que requiera desinstalar la aplicación lo que debe de hacer es lo siguiente, primero remover los paquetes utilizando el instalador, ejecute *MinGW intallation manager*, en la sección de paquetes elija la opción todos los paquetes y marquelos para remover de uno a uno:



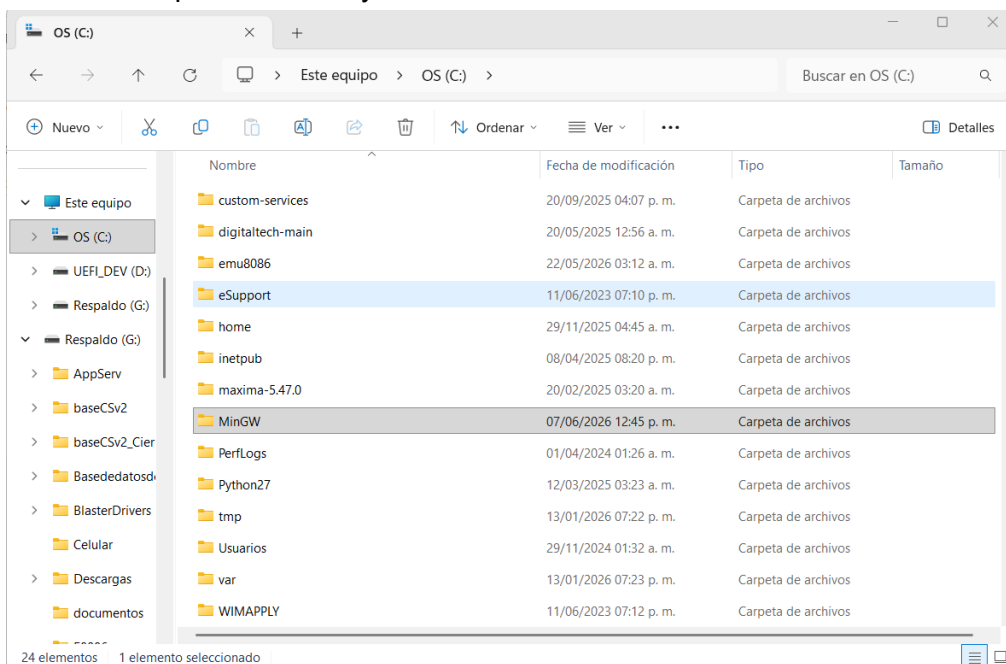
Después en el menú superior en la pestaña de instalación elija *Apply Changes*, espere a que se remuevan todos los paquetes.



Nota: asegúrese de marcar todos los paquetes en (A)

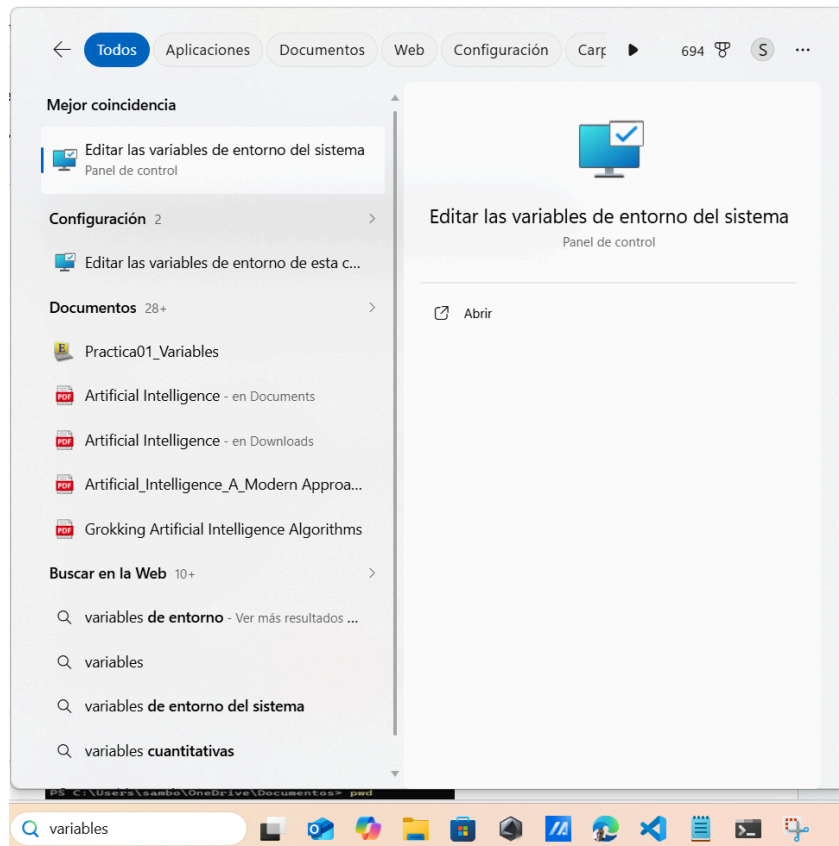


Para la siguiente etapa borre los archivos restantes de la instalación, busque la ubicación donde se instalaron los paquetes (usualmente en el directorio *C:\Program Files (x86)\MinG*), elimine la carpeta con todo y archivos.

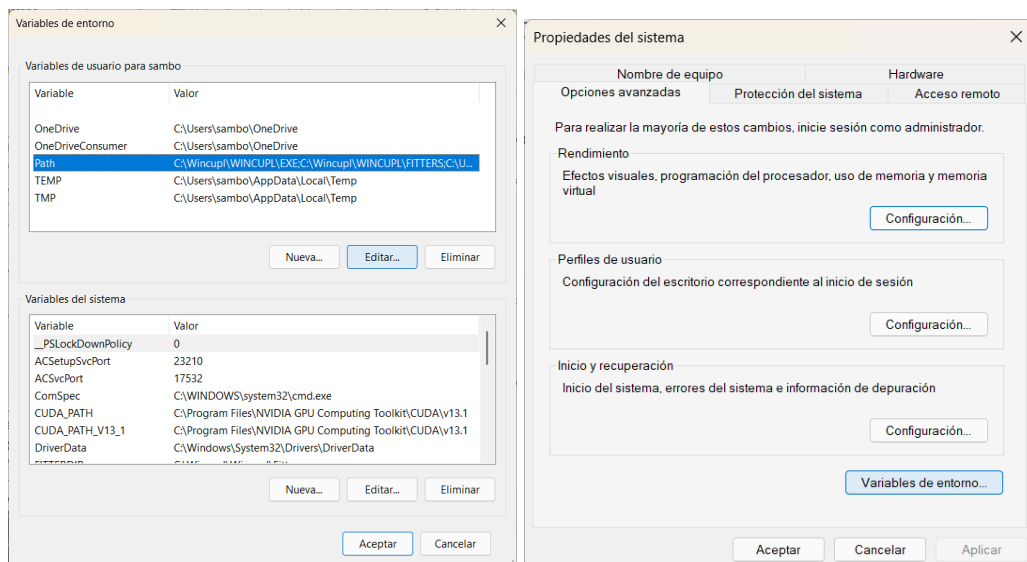


Nota: la ubicación de los archivos se indica en el siguiente paso variables de ambiente.

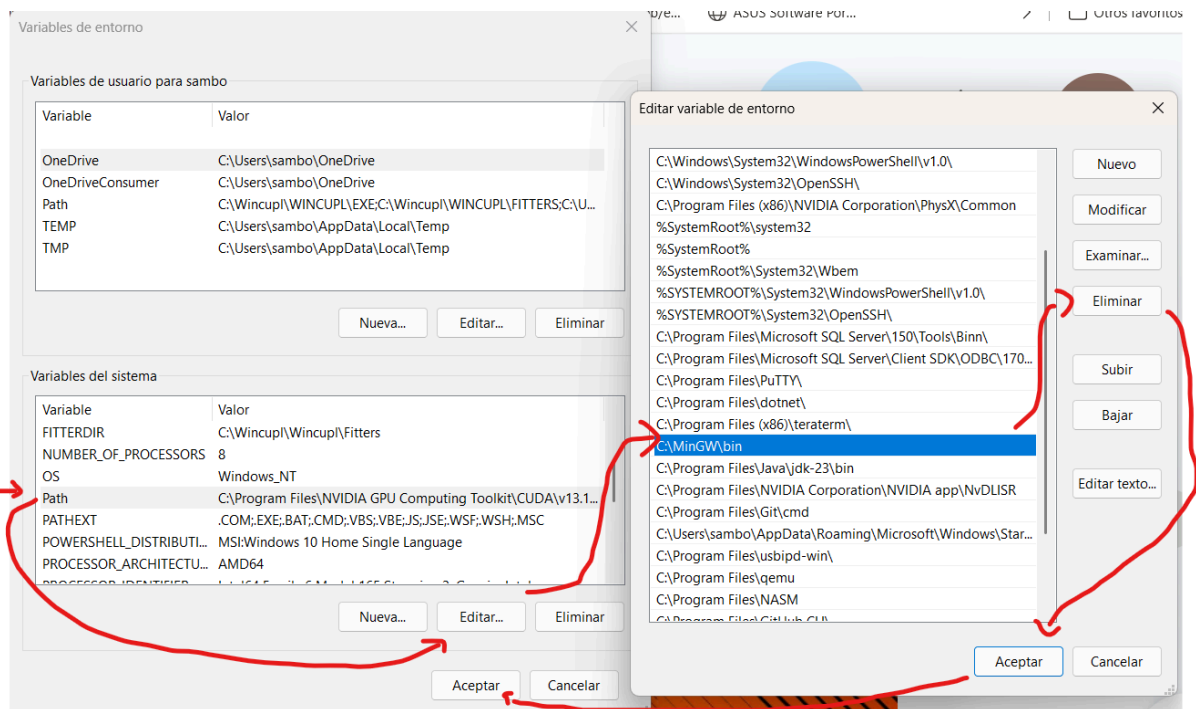
Por último remueva las variables de ambiente del programa, presione *windows + S* y escriba *variables de ambiente*



Elija el botón *editar variables de ambiente*, busque la opción *path* (ubicación) selecciónela y use el botón *Edit*



Busque las variables de ambiente relacionadas a MinGW (por ejemplo *c:\MinGW\bin*) y elija *Delete*, Presiones *OK* para guardar los cambios:



Para terminar elimine todos los accesos directos (*shortcuts*) y reinicie el equipo.

Anexo D

Codificando y Compilando con Vim y MinGW en la terminal

Nos movemos a una carpeta con permisos de edición y lanzamos el comando `vim <nombre>.c`

```
PS C:\Users\sambo\OneDrive\Documentos> pwd
Path
----
C:\Users\sambo\OneDrive\Documentos
PS C:\Users\sambo\OneDrive\Documentos> vim Hola.c
```

Y escribimos el siguiente código:

```
Hola.c (~\OneDrive\Documen x + v)
#include <stdio.h>
int main()
{
    printf("Hello World");
    return 0;
~
-- INSERTAR --
```

Para iniciar la edición presione la tecla 'i' y verifique que en la parte inferior indique INSERTAR.

Para salir y guardar presione la tecla 'esc' y cuando la barra inferior cambie, escriba ':x'

```
Hola.c (~\OneDrive\Documen x + v)
#include <stdio.h>
int main()
{
    printf("Hello World");
    return 0;
~
:x
```

Presione la tecla Enter para terminar, en la Terminal de PowerShell escriba lo siguiente para compilar el código `gcc hola.c -o hola.exe`, después verifique la creación del archivo usando `ls Hola*` (o *dir Hola**), ejecute el programa con `.\Hola.exe`, :

```
Administrador: Windows PowerShell
PS C:\Users\sambo\OneDrive\Documentos> gcc Hola.c -o Hola.exe
PS C:\Users\sambo\OneDrive\Documentos> ls Hola*

Directorio: C:\Users\sambo\OneDrive\Documentos

Mode                LastWriteTime         Length Name
----                -
-a---1            02/11/2024  02:27 a. m.             75 Hola.c
-a---1            02/11/2024  02:31 a. m.          40766 Hola.exe

PS C:\Users\sambo\OneDrive\Documentos> .\Hola.exe
Hello World
PS C:\Users\sambo\OneDrive\Documentos>
```

Nota: no olvide ingresar a la terminal de PowerShell como administrador
Para compilar programas codificados en lenguaje C++ escriba un archivo en Vim
Vim Hola.cpp y escriba lo siguiente:

```
Hola.cpp (~\OneDrive\Documentos) - VIM
#include <iostream>
using namespace std;
int main()
{
    cout << "Hola mundo"<<endl;
    return 0;
}
```

Compile, liste y ejecute el archivo generado como se muestra en la figura:

```
Administrador: Windows PowerShell
PS C:\Users\sambo\OneDrive\Documentos> vim Hola.cpp
PS C:\Users\sambo\OneDrive\Documentos> g++ Hola.cpp -o Hola_C.exe
PS C:\Users\sambo\OneDrive\Documentos> dir Hola*

Directorio: C:\Users\sambo\OneDrive\Documentos

Mode                LastWriteTime         Length Name
----                -
-a---1            02/11/2024  02:27 a. m.             75 Hola.c
-a---1            02/11/2024  02:39 a. m.            103 Hola.cpp
-a---1            02/11/2024  02:31 a. m.          40766 Hola.exe
-a---1            02/11/2024  02:45 a. m.          44550 Hola_C.exe
-a---1            02/11/2024  02:40 a. m.          44550 Hola_Cmas.exe

PS C:\Users\sambo\OneDrive\Documentos> .\Hola_C.exe
Hola mundo
PS C:\Users\sambo\OneDrive\Documentos>
```

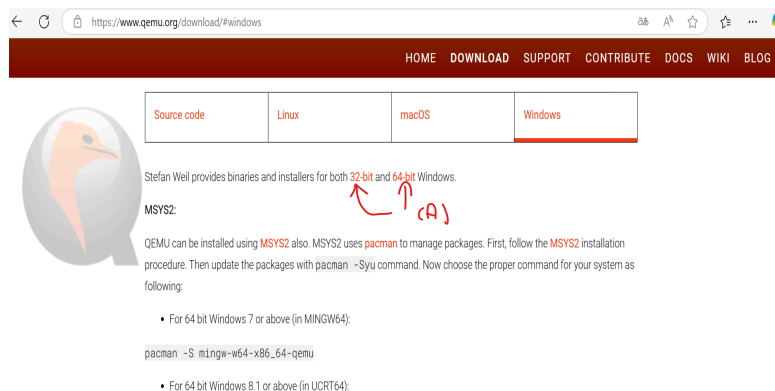
Note que esta vez se utilizó *dir Hola** para listar los archivos en lugar de *ls Hola**, ambos comandos realizan la misma función.

Anexo E

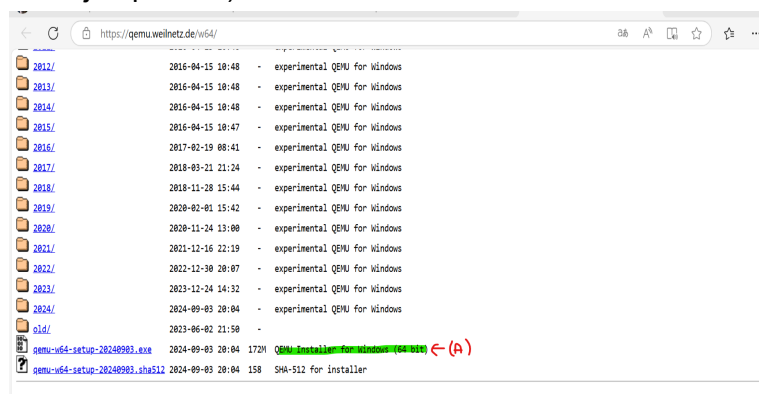
Máquinas Virtuales con Qemu

Usaremos el software Qemu para emular nuestras máquinas virtuales, en el explorador de internet, busque Qemu y ubique la sección Download, para la descarga del programa:

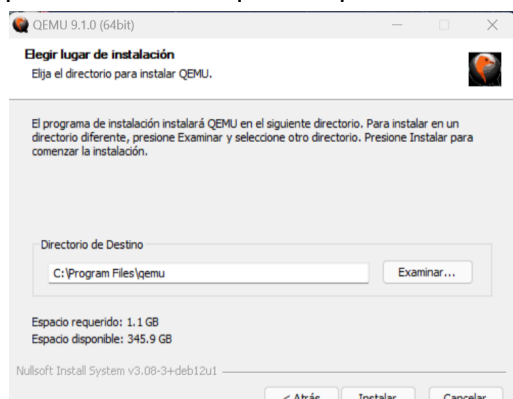
[Download QEMU - QEMU](#)



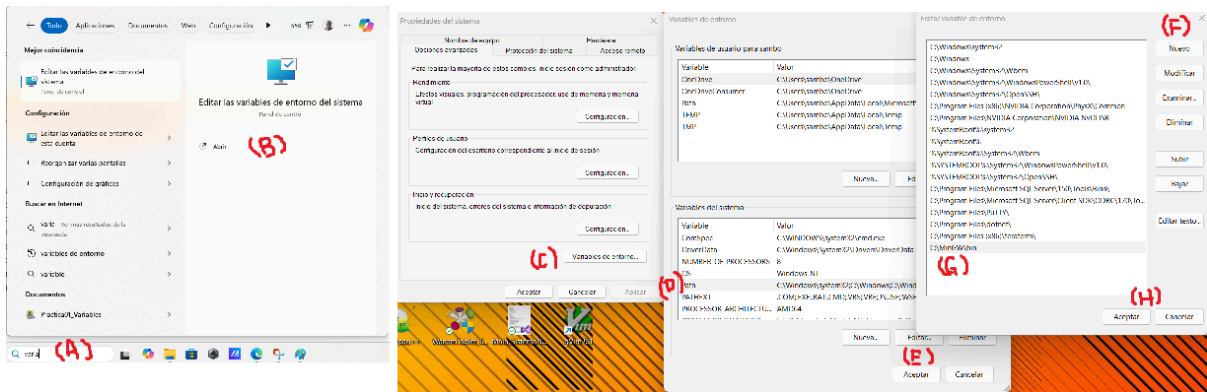
Como muestra (A) seleccione el instalador dependiendo del tipo de sistema que tenga (para este ejemplo 64x):



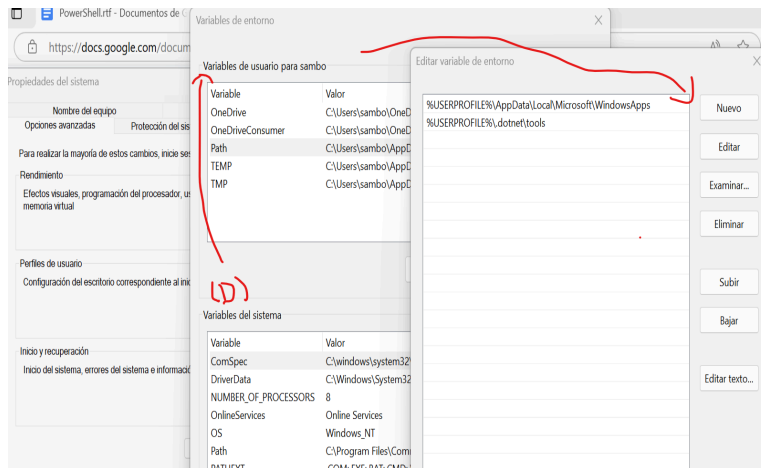
Nos redirecciona a una página con las descargas, elegimos el instalador de Window (A), para evitar tener que compilar e instalar el software.



No olvide tomar nota de la dirección de instalación del software, ya que necesitaremos agregarla a las variables de ambiente para completar la instalación, en el cuadro de búsqueda de Windows, escribimos *Variables de Ambiente* (A), ejecutamos la aplicación (B), presionamos el botón *Variables de Entorno* (C), elegimos del cuadro de texto la opción *Path* (D), en la ventana el botón (F) y en (G) pegamos la dirección y presionamos el botón (H) para terminar (damos clic en Aceptar en las ventanas anteriores).



Hacemos lo mismo para el bloque usuarios arriba del inciso (D):



Y pegamos la dirección del software Qemu, de esta manera podemos acceder al ejecutable desde cualquier localidad en el sistema. verificamos la instalación con el comando:

qemu-system-x86_64 --version

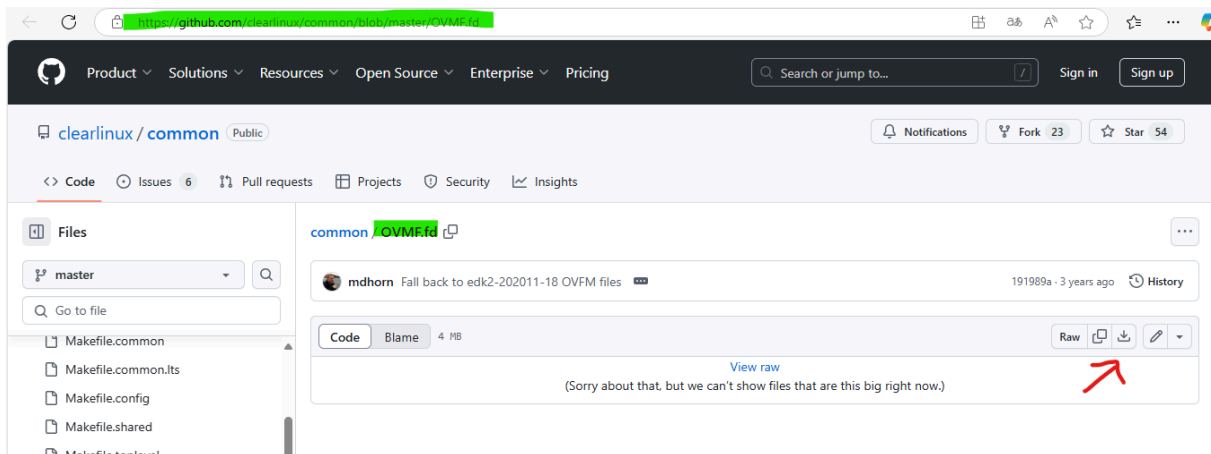
```

Administrador: Windows PowerShell

PS C:\Users\sambo\OneDrive\Documentos> qemu-system-x86_64 --version
QEMU emulator version 9.1.0 (v9.1.0-12064-gc658eebf44)
Copyright (c) 2003-2024 Fabrice Bellard and the QEMU Project developers
PS C:\Users\sambo\OneDrive\Documentos>
  
```

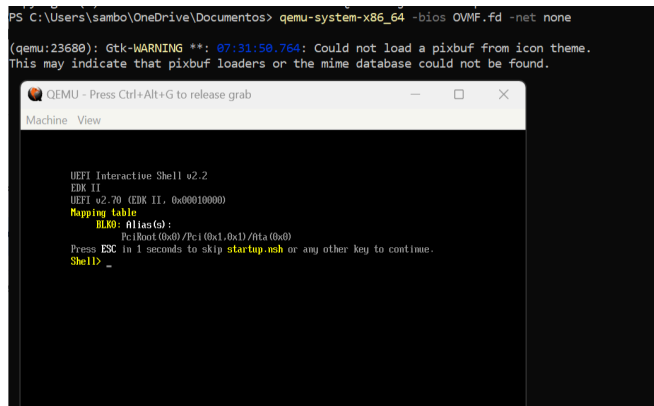
Para ejecutar una máquina virtual Qemu, usaremos el UEFI-BIOS de prueba, en su explorador de internet busque OVMF.fd download:

[common/OVMF.fd at master · clearlinux/common · GitHub](https://github.com/clearlinux/common/blob/master/OVMF.fd)



Descargue el archivo OVMF.fd usando el icono de descarga, posteriormente ejecutar Qemu con la ubicación del archivo de arranque:

qemu-system-x86_64 -bios OVMF.fd -net none

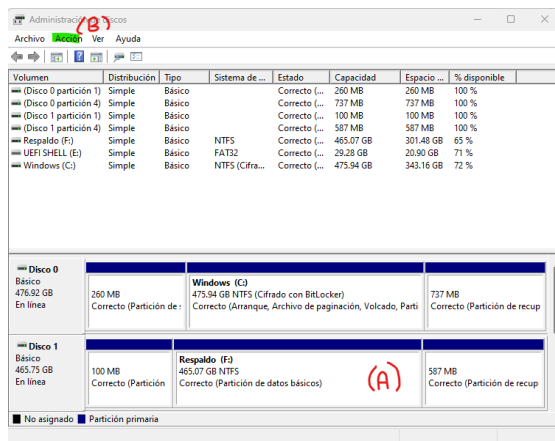


Generará una ventana de una VM ejecutando el Firmware UEFI-BIOS, para manipular cómodamente la ventana use la combinación de teclas *ctrl + alt + g*, esto libera el menú superior (la combinación de teclas se muestra en el marco superior de la ventana). para salir escriba el comando *reset -s* y enter.

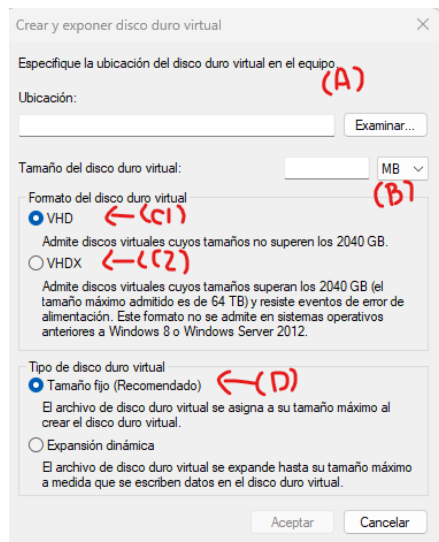
Anexo F

Agregando un disco USB virtual con FAT32 a nuestra máquina virtual

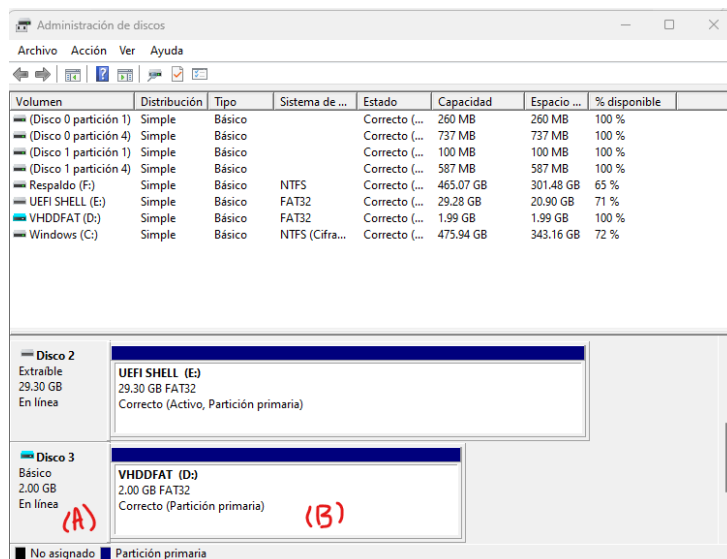
Crearemos un disco virtual para conectar con nuestra máquina virtual y poder ejecutar las aplicaciones UEFI que vayamos creando, usando la herramienta *disk management* (escriba la sentencia en buscador de la barra de herramientas de windows):



Seleccionamos el disco (A) y en el menú Accion->Crear VHD, nos envía a la ventana de configuración:



Donde especificamos en (A) la ubicación de nuestro disco virtual y en (B) el tamaño en Megas o en Gigas dependiendo de su fijo, seleccionamos (C1) para un disco de máximo 2Gb de espacio (use C2 si desea un disco más grande), importante indicar (D) para que el disco conserve su tamaño y respete el espacio asignado.

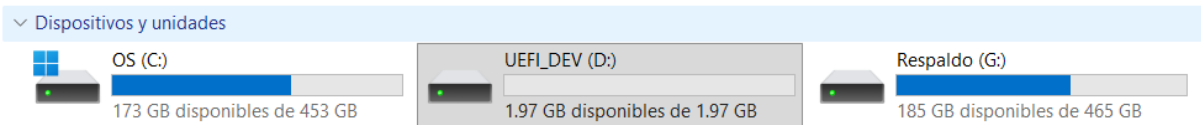


Cuando el disco se muestre, lo seleccionamos en (A) y hacemos click con el botón derecho para seleccionar "Nuevo Volumen" (Initialize Disk), importante escoger formato "MBR", asignar nombre a la unidad y volumen, posteriormente en (B) usar la opción "Nuevo Volumen" ("New Simple Volume"), prestamos atención a seleccionar como Formato FAT32, continuar con la instalación.

Asegure que el disco no está listado por el explorador de windows, con esto ya contamos con un disco virtual para conectar a Qemu:

 efi.c	02/02/2024 12:03 p. m.	C Source	1 KB
 efi.h	21/01/2024 11:45 p. m.	Archivo H	4 KB
 Hello_muestra.EFI	21/01/2024 11:45 p. m.	Archivo EFI	6 KB
 UEFI_VFat32	06/06/2026 11:38 p. m.	Archivo de image...	2,097,153 ...

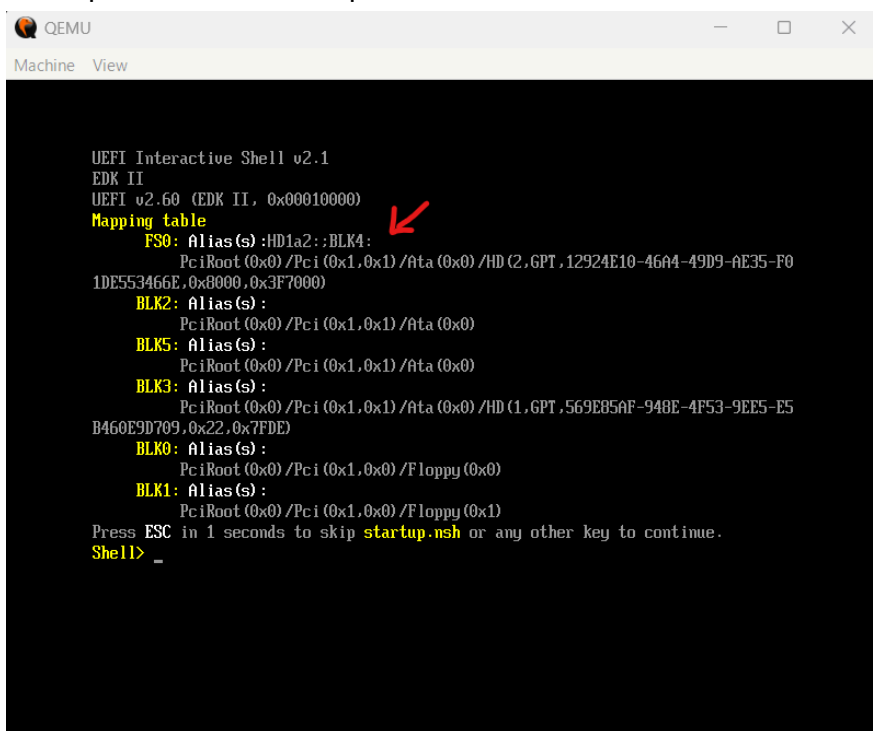
Cuando ingresemos a explorar el archivo este será detectado (montará) en el sistema de archivos y aparecerá como un disco más.



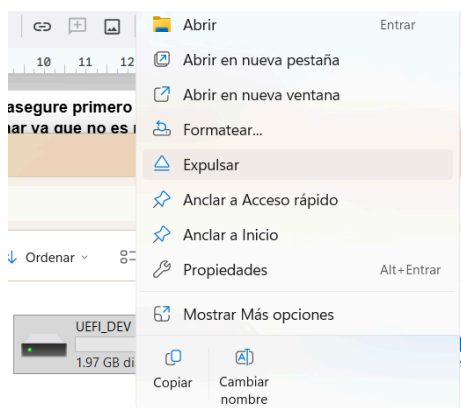
Ahora para poder conectar nuestro disco virtual con nuestra sesión de Qemu debemos expulsarlo del explorador para que se pueda redirigir a nuestra sesión con el siguiente comando :

```
qemu-system-x86_64 -bios OVMF.fd -net none -hda .\UEFI_VFat32.vhd
```

Ahora podemos observar que nuestra sesión ha detectado nuestro disco virtual

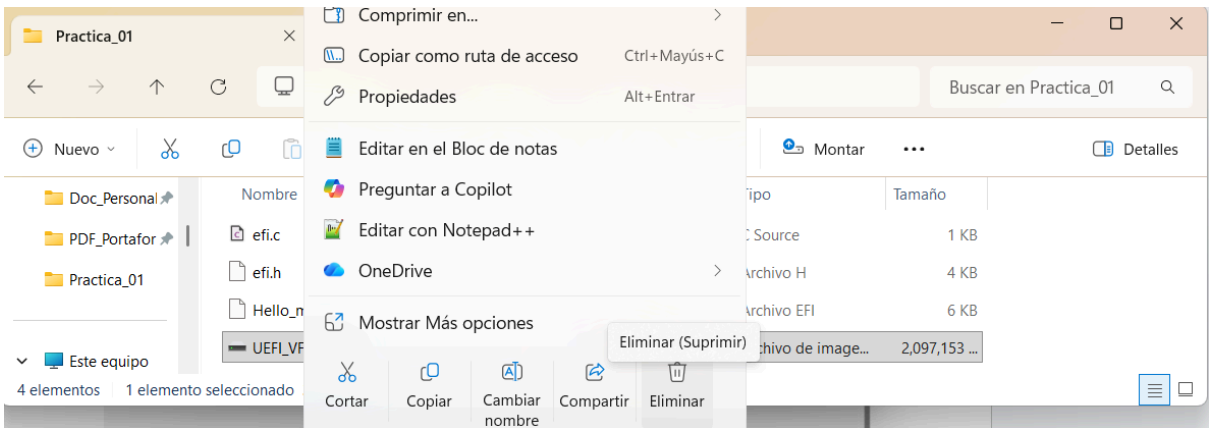


Si por alguna razón necesita eliminar el disco virtual **asegure primero de respaldar la información que contiene el disco antes de eliminar ya que no es recuperable**, con su respaldo ya creado utilice el siguiente procedimiento, primero expulse (desmonte) el disco del sistema:



Vaya a la ubicación del archivo del disco virtual en el explorador para completar la operación de eliminación (para volver a conectar el disco virtual, simplemente entre al disco en el explorador de windows en caso de que necesite respaldar información antes de borrar)

cuando no se liste el disco nos permitirá eliminar el volumen virtual seleccionando el disco en el explorador y dando click al botón derecho del ratón:



Elegimos la opción “Eliminar”, esto borrara al disco del sistema, **antes de eliminar asegure contar con respaldo del contenido.**

Anexo E

Compilando archivos para ejecutarse en ambiente UEFI

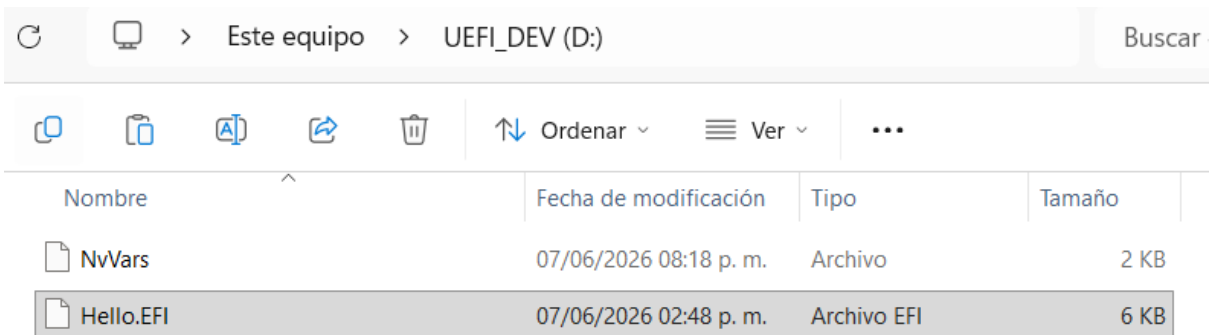
Compilamos una aplicación de muestra, para esto necesitamos los siguientes archivos:

 efi.c	02/02/2024 12:03 p. m.	C Source	1 KB
 efi.h	21/01/2024 11:45 p. m.	Archivo H	4 KB

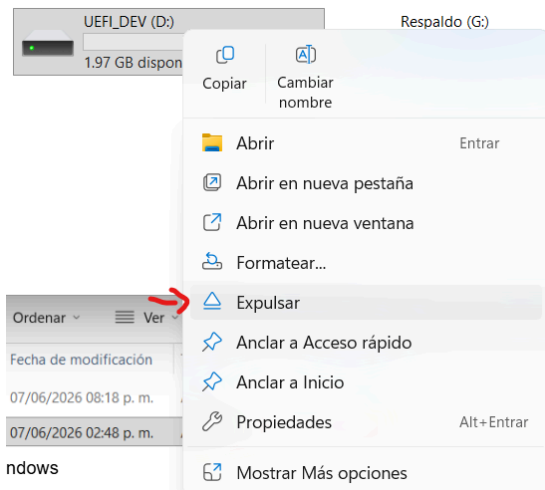
El contenido de los archivos anteriores se encuentra listado en los siguientes anexos.
El comando de comando de compilación es el siguiente:

```
$xargList =  
@('efi.c','-o','Hello.EFI','-nostdlib','-ffreestanding','-fshort-wchar','-mno-red-zone','-Wall','-Wextra','-Wpedantic','-std=c11','-Wl,--subsystem,10','-Wl,-e,efi_main','-Wl,--image-base,0'); & gcc  
@xargList
```

Este toma el archivo efi.c y genera el archivo Hello.EFI, el cual podemos copiar a nuestro disco virtual:

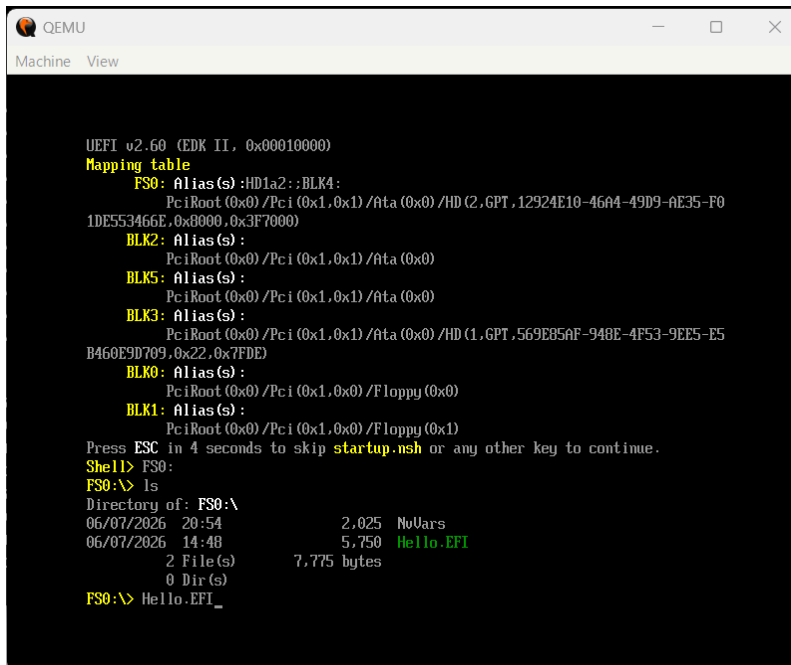


Nota: no elimine el archivo NrVars, es requerido para guardar las variables de entorno de la simulación.
El cual expulsamos del explorador de Windows

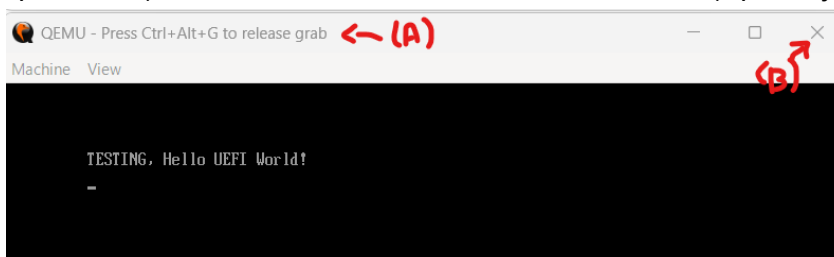


Para poder conectarlo a nuestra sesión Qemu con bios UEFI para poder verificar si funciona correctamente ejecutamos el siguiente comando en la terminal de PowerShell:

`qemu-system-x86_64 -bios OVMF.fd -net none -hda .\UEFI_VFat32.vhd`



Posteriormente liste los archivos dentro del directorio y escriba el nombre de nuestra aplicación (convenientemente mostrada en color verde), para ejecutarla:



Veremos un mensaje en pantalla como se muestra en la imagen anterior, cómo nuestra aplicación de ejemplo no cuenta con código para regresar al shell UEFI para poder salir de la sesión de Qemu debemos presionar las teclas Ctrl + Alt + G como muestra (A) y presione el icono X de la ventana para salir.

Anexo F

Contenido de archivos efi.c y efi.h

efi.c

```
//
#include "efi.h"

//entry point set up
EFI_STATUS efi_main(EFI_HANDLE ImageHandle, EFI_SYSTEM_TABLE
*SystemTable) {
    //TODO: Remove this line when using input params
    (void)ImageHandle, (void)SystemTable; //prevent compiler warnings

    // Reset Console Output
    SystemTable->ConOut->Reset(SystemTable->ConOut, false);

    //CLEAR console output (clear screen to background color and
    // set cursor to 0.0
    SystemTable->ConOut->ClearScreen(SystemTable->ConOut);

    // Write String
    SystemTable->ConOut->OutputString(SystemTable->ConOut, u"TESTING,
Hello UEFI World!\r\n"); // power2(3, 5)
    //"3 times 2 to the power of 5 is %d\n", power2(3, 5)

    // Infinite Loop
    while (1);

    return EFI_SUCCESS;
}
```

efi.h

```
// NOTE: "VOID *" Fields in structs = NOT implemented!!
// They are defined on the UEFI specs, but they are not
// implemented YET
#include <stdint.h>
#include <stdbool.h>

#if __has_include(<uchar.h>)
    #include <uchar.h>
#endif

//UEFI Spec 2.10 section 2.4
#define IN          //visual helpers to remark the flow of the data
```

```

#define OUT          //this is not has any meaning they are Vacum CONST
#define OPTIONAL
#define CONST const

//EFIAPI defines the calling conventions for EFI defined functions
#define EFIAPI __attribute__((ms_abi)) //x86_64 Microsoft calling
convention

//Data types UEFI Spec 2.10 sections 2.3
//typedef uint8_t UNIT8;
typedef uint8_t      BOOLEAN;    //0 = False 1 = True
typedef int64_t      INTN;
typedef uint64_t     UINTN;
typedef int8_t       INT8;
typedef uint8_t      UINT8;
typedef int16_t      INT16;
typedef uint16_t     UINT16;
typedef int32_t      INT32;
typedef uint32_t     UINT32;
typedef int64_t      INT64;
typedef uint64_t     UINT64;
typedef char         CHAR8;

//UTF-16 Equivalent is type, for UCS-2 characters
//codepoints <= 0xFFFF_FFFF
#ifndef _UCHAR_H
    typedef uint_least16_t char16_t;
#else
    typedef char16_t CHAR16;
#endif
typedef char16_t CHAR16;

typedef void VOID;

typedef struct EFI_GUID{    //look for EFI_GUID on specs UEFI APPENDIX
A
    UINT32      TimeLow;
    UINT16      TimeMid;
    UINT16      TimeHighAndVersion;
    UINT8       ClockSeqHighAndReserved;
    UINT8       ClockSeqLow;
    UINT8       Node[6];
}__attribute__((packed)) EFI_GUID;

```



```

typedef UINTN          EFI_STATUS;
typedef VOID           *EFI_HANDLE;
typedef VOID           *EFI_EVENT;
typedef UINT64         EFI_LBA;
typedef UINTN          EFI_TPL;

//EFI_STATUS Codes - UEFI Spec 2.10 Appendix D
#define EFI_SUCCESS 0ULL

//TODO: Add EFI_ERROR() macro/other for checking if EFI_STATUS
//high bit is set, if so it is an ERROR status

//EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL
typedef struct EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL
EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL;

//EFI_SIMPLE_TEXT_RESET_PROTOCOL: UEFI Spec 2.10 section 12.4.1
typedef
EFI_STATUS
(EFIAPI *EFI_TEXT_RESET) (
    IN EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL    *This,
    IN BOOLEAN                            ExtendedVerification
);

//EFI_SIMPLE_TEXT_STRING_PROTOCOL: UEFI Spec 2.10 section 12.4.1
typedef
EFI_STATUS
(EFIAPI *EFI_TEXT_STRING) (
    IN EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL    *This,
    IN CHAR16                             *String
);

//EFI_SIMPLE_TEXT_CLEAR_SCREEN_PROTOCOL: UEFI Spec 2.10 section 12.4.1
typedef
EFI_STATUS
(EFIAPI *EFI_TEXT_CLEAR_SCREEN) (
    IN EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL    *This
);

//EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL: UEFI Spec 2.10 section 12.4.1
typedef struct EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL{
    EFI_TEXT_RESET                Reset;

```

```

    EFI_TEXT_STRING          OutputString;
    void                      *TestString;
    void                      *QueryMode;
    void                      *SetMode;
    void                      *SetAttribute;
    EFI_TEXT_CLEAR_SCREEN    ClearScreen;
    void                      *SetCursorPosition;
    void                      *EnableCursor;
    void                      *Mode;
} EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL;

// EFI_TABLE_HEADER: UEFI Spec 2.10 section 4.2.1
typedef struct{
    UINT64 Signature;
    UINT32 Revision;
    UINT32 HeaderSize;
    UINT32 CRC32;
    UINT32 Reserved;
} EFI_TABLE_HEADER;

//EFI_SYSTEM_TABLE:
typedef struct{
    EFI_TABLE_HEADER          Hdr;
    CHAR16                    *FirmwareVendor;
    UINT32                    FirmwareRevision;
    EFI_HANDLE                ConsoleInHandle;
    VOID                      *ConIn;
//EFI_SIMPLE_TEXT_INPUT_PROTOCOL
    EFI_HANDLE                ConsoleOutHandle;
    EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL *ConOut; //VOID
*Conout;
    EFI_HANDLE                StandardErrorHandle;
    VOID                      *StdErr;
//EFI_SIMPLE_TEXT_OUTPUT_PROTOCOL
    VOID                      *RuntimeServices;
//EFI_RUNTIME_SERVICES
    UINTN                     NumberOfTablesEntries;
    VOID                      *ConfigurationTable;
//EFI_CONFIGURATION_TABLE
} EFI_SYSTEM_TABLE;

//EFI_IMAGE_ENTRY_POINT: UEFI Spec 2.10 section 4.1.1

```

```
typedef EFI_STATUS                                     //efi.h:69:1: warning: 'ms_abi'
attribute directive ignored [-Wattributes]
(EFIAPI *EFI_IMAGE_ENTRY_POINT) (                     //check the MICROSOFT CALLING
CONVENTIONS on Wiki
    IN EFI_HANDLE ImageHandle,
    IN EFI_SYSTEM_TABLE *SystemTable                 //Note IN is only meaning the
flow of the information
);
```

Bibliografía:

Página proyecto tianocore

<https://www.tianocore.org>

Página de OVMF.fd

[OVMF · tianocore/tianocore.github.io Wiki](https://tianocore.github.io Wiki)

Página para descargar la especificación UEFI:

uefi.org/sites/default/files/resources/UEFI_Spec_2_10_Aug29.pdf